Extensible/Rule Based Query Rewrite Optimization in Starburst

Hamid Pirahesh
Joseph M. Hellerstein*
Waqar Hasan†

IBM Almaden Research Center
San Jose, CA 95120, USA
pirahesh@almaden.ibm.com, joey@cs.berkeley.edu, hasan@hpl.hp.com

Abstract

This paper describes the Query Rewrite facility of the Starburst extensible database system, a novel phase of query optimization. We present a suite of rewrite rules used in Starburst to transform queries into equivalent queries for faster execution, and also describe the production rule engine which is used by Starburst to choose and execute these rules. Examples are provided demonstrating that these Query Rewrite transformations lead to query execution time improvements of orders of magnitude, suggesting that Query Rewrite in general — and these rewrite rules in particular — are an essential step in query optimization for modern database systems.

1 Introduction

In traditional database systems, query optimization typically consists of a single phase of processing in which access methods, join orders and join methods are chosen to provide an efficient plan for executing a user's declarative query. We refer to this phase as *plan optimization*. In this paper we present a distinct phase of query optimization, Query Rewrite, which has been implemented in the Starburst DBMS [HCL+90] to precede plan optimization in the processing of a query.

The goal of Query Rewrite is twofold:

1. *Make queries as declarative as possible:* In database languages such as SQL, it is often possible for a poorly expressed query, though ostensibly declarative, to force typical plan optimizers into choosing sub-optimal execution plans. A major goal of Query Rewrite is the transformation of such "procedural" queries into equivalent but more declarative queries.
2. *Perform natural heuristics:* Certain heuristics can be performed in Query Rewrite and are generally accepted in the literature as being valuable. A typical example is that of "predicate push-down", in which predicates are applied as early as possible in the query (*i.e.* they are "pushed" from their original positions into table accesses, subqueries, views, etc.) Such rules can significantly improve query execution time, and while a few of these heuristics are done in typical plan optimizers, they often can be applied in a more general way during Query Rewrite.

Although it is accepted doctrine that query languages should be declarative, we shall see in our examples that alternative but equivalent expressions of a query can have widely varying performance, often differing by orders of magnitude. It is therefore our conviction that Query Rewrite is an essential step in query optimization, since it further ensures that the expression of the query will be insignificant with respect to its performance.

1.1 Queries with Path Expressions

The goals of Query Rewrite explained above are even more significant in Object Oriented applications, which typically generate complicated queries with "path expressions" connecting various collections of objects [BTA90, LLOW91, LLPS91]. In such applications, both the *complexity of the logic* and the *volume of the data* are far greater than in traditional DBMS applications [HSS88]. As a result, query optimization becomes increasingly important.

The following is an example of a query involving path expressions, using the Object SQL syntax defined in [BTA90].[1] This query is a small variation of an example presented in [BTA90]. The example database contains records of patients. Medical records are *set attributes* of patients. All accesses to data are via methods. Given a patient's record, the medical records are returned by the function *get_medical_records*. The example retrieves male patients who have been diagnosed with malaria or smallpox prior to '10/10/89'. The *FROM* clause enumerates the patients and the *WHERE* clause restricts the patients to males and checks for the existence of a malaria or smallpox diagnosis prior to a given date.

```
SELECT DISTINCT P
FROM Patient p  IN Patient_Set
WHERE p.sex == 'male' &&
  EXISTS ( SELECT r
           FROM Medical_record r  IN p.get_medical_record()
           WHERE r.get_date() < '10/10/89' &&
             ( r.get_diagnosis() == 'Malaria' ||
               r.get_diagnosis() == 'Smallpox' );
```

Queries such as this essentially involve path expressions in which, given a record, the related information is obtained through a path (*e.g.*, getting the medical records of a patient). Queries involving path expressions are very common in complex applications such as CAD/CAM. In general, many path expressions may be involved in a query, and each may have a length of more than one. Finding an efficient execution plan for such path expressions is a problem very similar to that of optimizing (nested) SQL subqueries. The above query is commonly executed by enumerating the male patients,

*Current address: CS Division, Department of EECS, University of California, Berkeley, CA 94720

†Current address: Hewlett-Packard Laboratories, Palo Alto, CA 94305

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

1992 ACM SIGMOD - 6/92/CA, USA
© 1992 ACM 0-89791-522-4/92/0005/0039...$1.50

[1] This is a proposal for a standard OO query language, and is implemented by some commercial OODBMSs